Módulo 8 - Aulas 3 e 4

Tarefas

No final deste módulo você deve submeter em um ÚNICO arquivo PDF os seguintes prints:

- Atividade 8.6: passo 7.
- Atividade 8.7: passo 5.
- Atividade 8.8: passo 5.
- Atividade 8.9: passo 6.
- Atividade 8.10: passo 9.

Regras para a elaboração do documento:

1. Antes de cada Print, adicione obrigatoriamente uma frase explicativa que sinalize do que se trata o print. A inserção de prints sem a devida frase explicativa será considerada como tentativa de atrapalhar a correção do instrutor e será penalizada a critério do instrutor. Exemplo:

Print da atividade 8.6: [Imagem com o Print]

2. Os Prints devem ser tirados da TELA CHEIA. Quando tirados da VM da AWS, devem ser capturados **obrigatoriamente** da tela cheia clicando no botão "Screenshot" → "Take screenshot" da barra de ferramentas do Hypervisor da AWS.
3. Insira **somente a quantidade de Prints solicitados por atividade** usando exclusivamente 1 página por print. **A página do documento onde você vai inserir o Print deve estar com a orientação no modo PAISAGEM** ´para termos melhor aproveitamento do espaço. Ou seja, seu documento deverá ter a mesma quantidade de páginas que a quantidade do total de Prints! A inserção de prints desnecessários será considerada como tentativa de atrapalhar a correção do instrutor e será penalizada com nota 0.

4. Apresente Prints legíveis e com tamanho correto para fácil leitura. O envio de prints com letras minúsculas poderá ser considerado como tentativa de atrapalhar a correção do instrutor e será penalizada a critério do instrutor.

Atividade 8.6 – Computando o Hash MD5, RIPEMD-160 e Blake2 de um arquivo no Kali Linux

Nesta atividade, vamos usar o Terminal para descobrir como obter o Hash dos algoritmos MD5, RIPEMD-160 e Blake2 de um arquivo qualquer no Kali Linux.

Vamos começar inicializando nosso Kali Linux via RDP ao IP: 192.168.98.40, com usuário “aluno” e senha “rnpesr”.

1. Abra o Terminal e execute o seguinte comando (com senha “rnpesr”) para ser super usuário:

```
┌──(aluno㉿kali)-[~]
└─$ sudo -i
[sudo] senha para aluno:
```

2. Acesse a pasta Arquivos e crie um arquivo de texto chamado “mensagem.txt”:

```
┌──(root㉿kali)-[~]
└─# cd /home/aluno/Documentos/Arquivos

┌──(root㉿kali)-[/home/aluno/Documentos/Arquivos]
└─# nano mensagem.txt
```

3. Insira o seguinte texto:

```
Hackers do bem!
```

Em seguida, aperte “Ctrl + x”, “s” para salvar e “Enter” para confirmar. Neste momento, acabamos de criar o arquivo “mensagem.txt”.

4. Verifique que o texto criado está presente na pasta:

```
┌──(root㉿kali)-[/home/aluno/Documentos/Arquivos]
└─# ls
mensagem.txt
```

5. Agora, vamos encontrar o valor do Hash MD5 do arquivo “mensagem.txt”:

```
┌──(root㉿kali)-[/home/aluno/Documentos/Arquivos]
└─# md5sum mensagem.txt
ac640793b0fe42e14c071f53fe8f8486  mensagem.txt
```

O comando irá processar o arquivo e calcular o resumo Hash. Ele exibirá uma sequência de caracteres hexadecimais de 32 dígitos, que representa o resumo Hash MD5 do arquivo.

6. Encontremos o valor do Hash RIPEMD-160 do arquivo “mensagem.txt”:

```
┌──(root㉿kali)-[/home/aluno/Documentos/Arquivos]
└─# openssl dgst -ripemd160 mensagem.txt
RIPEMD-160(mensagem.txt)= 1ec06a94e04fc961bad8957e1fd1f27af9dd421d
```

O comando irá processar o arquivo e calcular o resumo Hash. Ele exibirá uma sequência de caracteres hexadecimais de 40 dígitos, que representa o resumo Hash RIPEMD-160 do arquivo.

7. Finalmente, vamos encontrar o valor do Hash Blake2 do arquivo “mensagem.txt” (**NÃO SE ESQUEÇA DE PRINTAR ESTE PASSO!**):

```
┌──(root㉿kali)-[/home/aluno/Documentos/Arquivos]
└─# b2sum mensagem.txt
a7158864d19a20e26c1bf13b1a802b73614932d52cea5282e8b0bbc6b6520f322510b1e
747a1514429b42a405dc8403c845382859a2b59e34784c736effc3c98  mensagem.txt
```

O comando processará o arquivo e calculará o resumo Hash. Ele exibirá uma sequência de caracteres hexadecimais de 64 dígitos, que representa o resumo Hash Blake2 do arquivo.

8. Não apague o arquivo “mensagem.txt”, somente feche o Terminal.

Parabéns! Agora você poderá calcular o Hash com qualquer desses algoritmos em cima de qualquer tipo de arquivo!

Atividade 8.7 – Computando o Hash da família SHA (SHA-1, SHA-2 de 512 bits e SHA-3 de 512 bits) de um arquivo no Kali Linux

Nesta atividade, vamos usar o Terminal para descobrir como obter o Hash dos algoritmos SHA-1, SHA-2 de 512 bits e SHA-3 de 512 bits de um arquivo qualquer no Kali Linux.

Vamos começar inicializando nosso Kali Linux via RDP ao IP: 192.168.98.40, com usuário “aluno” e senha “rnpesr”.

1. Abra o Terminal e execute o seguinte comando (com senha “rnpesr”) para ser super usuário:

```
┌──(aluno㉿kali)-[~]
└─$ sudo -i
[sudo] senha para aluno:
```

2. Acesse a pasta Arquivos e verifique a presença do arquivo de texto chamado “mensagem.txt”:

```
┌──(root㉿kali)-[~]
└─# cd /home/aluno/Documentos/Arquivos

┌──(root㉿kali)-[/home/aluno/Documentos/Arquivos]
└─# ls
mensagem.txt
```

3. Agora, vamos encontrar o valor do Hash SHA-1 do arquivo “mensagem.txt”:

```
┌──(root㉿kali)-[/home/aluno/Documentos/Arquivos]
└─# sha1sum mensagem.txt
287f168af5acf44b3b06a07fb6f3a4e882a9baab  mensagem.txt
```

O comando irá processar o arquivo e calcular o resumo Hash. Ele exibirá uma sequência de caracteres hexadecimais de 40 dígitos, que representa o resumo Hash SHA-1 do arquivo.

4. Agora, vamos encontrar o valor do Hash SHA-2 de 512 bits do arquivo "mensagem.txt":

```
┌──(root㉿kali)-[/home/aluno/Documentos/Arquivos]
└─# sha512sum mensagem.txt
d0774282912bf4b5479165a9b316f23fea1f9ae3f620f400618813347a13c8d1e297fb5
1196def065dafe0aa21239d719f51a8cf58410bac22fc4b40b29efeab  mensagem.txt
```

O comando irá processar o arquivo e calcular o resumo Hash. Ele exibirá uma sequência de caracteres hexadecimais de 128 dígitos, que representa o resumo Hash SHA-2 de 512 bits do arquivo.

5. Finalmente, vamos encontrar o valor do Hash SHA-3 de 512 bits do arquivo "mensagem.txt" (**NÃO SE ESQUEÇA DE PRINTAR ESTE PASSO!**):

```
┌──(root㉿kali)-[/home/aluno/Documentos/Arquivos]
└─# sha3sum -b -a 512 mensagem.txt
07ad8830012228703f01d1961a759c907214a617c2c42fade253af6bf42bf4500a54038
f1fb960165e6cb4985fed141ed90b949f7915d2379c7a67d5484930f1 *mensagem.txt
```

O parâmetro -b é usado para mostrar o resultado em formato binário, e o parâmetro -a 512 indica que você deseja usar a variante de 512 bits do SHA-3. O comando irá processar o arquivo e calcular o resumo Hash. Ele exibirá uma sequência de caracteres hexadecimais de 128 dígitos, que representa o resumo Hash SHA-3 de 512 bits do arquivo.

6. Não apague o arquivo "mensagem.txt", somente feche o Terminal.

Parabéns! Agora você poderá calcular o Hash com qualquer desses algoritmos em cima de qualquer tipo de arquivo!

Atividade 8.8 – Implementando criptografia assimétrica para confidencialidade com RSA-2048 em um arquivo no Kali Linux

Nesta atividade, vamos usar o Terminal para cifrar e decifrar qualquer arquivo usando criptografia assimétrica com o algoritmo de RSA-2048 no Kali Linux.

Vamos começar inicializando nosso Kali Linux via RDP ao IP: 192.168.98.40, com usuário “aluno” e senha “rnpesr”.

1. Abra o Terminal e execute o seguinte comando (com senha “rnpesr”) para ser super usuário:

```
┌──(aluno㉿kali)-[~]
└─$ sudo -i
[sudo] senha para aluno:
```

2. Acesse a pasta Arquivos e verifique a presença do arquivo de texto chamado “mensagem.txt”:

```
┌──(root㉿kali)-[~]
└─# cd /home/aluno/Documentos/Arquivos

┌──(root㉿kali)-[/home/aluno/Documentos/Arquivos]
└─# ls
mensagem.txt
```

3. Agora, vamos gerar as chaves privada e pública do algoritmo RSA. No Kali Linux, você pode usar a ferramenta OpenSSL para isso. Na sequência, verifique que 2 arquivos foram criados:

```
┌──(root㉿kali)-[/home/aluno/Documentos/Arquivos]
└─# openssl genpkey -algorithm RSA -out chave_privada.pem
.+.........+........+...+.......+...........+...+++++++++++++++++++
++++++++++++++++++++++++++++++++++++++++++++*.....+.+......+......
+........+...+..+......+........+.+...+++++++++++++++++++++++++++++
++++++++++++++++++++++++++++++++*....+...........+.............
+.......+..+.......+...+.....+.........................+.+...
+..........+...+.+........+...+..+......+.......+.....+...+....
+........+.......+..+....+..+.........+.....++++++++++++++++++++++++
++++++++++++++++++++++++++++++++++++++
..+...+..+....+++++++++++++++++++++++++++++++++++++++++++++++++++++++
++++++++*..+...+...+............+..+.......+..+...+.......+........+++
+++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++*...+.
+..........+.......+...+........+.+...+...............+...+........
+.....+...+.+..+...+....+.....+.+......+.....+....+......+.....+...
+.........+..+........+...........+...+...........+....+...+.....
+.......+...+...+..+.+...................
+.............................+.......+........+......+....
+..........+......+....+...+...+...........+..+........+......+.+...
+..+...+........+......+.+.....................+.......+........
+.....+....+...+...+..+........+.........+.....+...+++++++++++++++++
++++++++++++++++++++++++++++++++++++++++++++

┌──(root㉿kali)-[/home/aluno/Documentos/Arquivos]
└─# openssl rsa -pubout -in chave_privada.pem -out chave_publica.pem
writing RSA key

┌──(root㉿kali)-[/home/aluno/Documentos/Arquivos]
└─# ls
chave_privada.pem  chave_publica.pem  mensagem.txt
```

Esses comandos gerarão um arquivo “chave_privada.pem” contendo a chave privada RSA e um arquivo “chave_publica.pem” contendo a chave pública RSA.

4. O próximo passo é cifrar o arquivo “mensagem.txt” com a chave pública gerada anteriormente e visualizar o arquivo cifrado gerado:

```
┌──(root㉿kali)-[/home/aluno/Documentos/Arquivos]
└─# openssl pkeyutl -encrypt -pubin -inkey chave_publica.pem -in
mensagem.txt -out arquivo_criptografado

┌──(root㉿kali)-[/home/aluno/Documentos/Arquivos]
└─# ls
arquivo_criptografado  chave_privada.pem  chave_publica.pem
mensagem.txt
```

Os parâmetros usados no comando para decifrar são:

- **pkeyutl**: Este é o comando do OpenSSL para operações de chave pública e privada.
- **-encrypt**: Especifica que a operação a ser realizada é a criptografia.
- **-pubin**: Indica que a chave fornecida é uma chave pública. Isso é importante para especificar o tipo de chave que está sendo usada na operação.
- **-inkey chave_publica.pem**: Especifica o arquivo contendo a chave pública a ser usada para criptografar os dados. "chave_publica.pem" é o nome do arquivo que contém a chave pública.
- **-in mensagem.txt**: Especifica o arquivo contendo os dados a serem criptografados. "mensagem.txt" é o nome do arquivo que contém a mensagem a ser criptografada.
- **-out arquivo_criptografado**: Especifica o arquivo de saída onde os dados criptografados serão salvos. "arquivo_criptografado" é o nome do arquivo onde os dados criptografados serão armazenados após o processo de criptografia.

5. Agora, vamos decifrar o arquivo cifrado no passo anterior “arquivo_criptografado” usando a chave privada “chave_privada.pem”. O resultado será armazenado em um novo arquivo chamado “mesagem_recuparada.txt”. Visualizaremos o resultado (**NÃO SE ESQUEÇA DE PRINTAR ESTE PASSO!**):

```
┌──(root㉿kali)-[/home/aluno/Documentos/Arquivos]
└─# openssl pkeyutl -decrypt -inkey chave_privada.pem -in
arquivo_criptografado -out mensagem_recuperada.txt

┌──(root㉿kali)-[/home/aluno/Documentos/Arquivos]
└─# ls
arquivo_criptografado  chave_privada.pem  chave_publica.pem
mensagem_recuperada.txt  mensagem.txt
```

Os parâmetros usados no comando são:

- **pkeyutl**: Comando do OpenSSL para operações de chave pública e privada.
- **-decrypt**: Indica que a operação a ser realizada é a descriptografia.
- **-inkey chave_privada.pem**: Especifica o arquivo contendo a chave privada a ser usada para descriptografar os dados. "chave_privada.pem" é o nome do arquivo que contém a chave privada.
- **-in arquivo_criptografado**: Especifica o arquivo contendo os dados criptografados a serem descriptografados. "arquivo_criptografado" é o nome do arquivo que contém os dados criptografados.
- **-out arquivo_descriptografado**: Especifica o arquivo de saída onde os dados descriptografados serão salvos. "arquivo_descriptografado" é o nome do arquivo onde os dados descriptografados serão armazenados após o processo de descriptografia.

6. Finalmente, vamos verificar se de fato conseguimos recuperar a mensagem original visualizando o arquivo “mesagem_recuparada.txt”.

```
┌──(root㉿kali)-[/home/aluno/Documentos/Arquivos]
└─# cat mensagem_recuperada.txt
Hackers do bem!
```

Confira que realmente a mensagem original foi recuperada!

7. Apague todos os arquivos com exceção do arquivo “mensagem.txt” e posteriormente feche o Terminal:

```
┌──(root㉿kali)-[/home/aluno/Documentos/Arquivos]
└─# rm arquivo_criptografado chave_privada.pem chave_publica.pem
mensagem_recuperada.txt

┌──(root㉿kali)-[/home/aluno/Documentos/Arquivos]
└─# ls
mensagem.txt
```

Parabéns! Agora você sabe usar o algoritmo de criptografia assimétrica RSA!

Atividade 8.9 – Implementando criptografia assimétrica para confidencialidade com ECC-256 em um arquivo no Kali Linux

Nesta atividade, vamos usar o Terminal para cifrar e decifrar qualquer arquivo usando criptografia assimétrica com o algoritmo de ECC-256 no Kali Linux.

Vamos começar inicializando nosso Kali Linux via RDP ao IP: 192.168.98.40, com usuário “aluno” e senha “rnpesr”.

1. Abra o Terminal e execute o seguinte comando (com senha “rnpesr”) para ser super usuário:

```
┌──(aluno㉿kali)-[~]
└─$ sudo -i
[sudo] senha para aluno:
```

2. Acesse a pasta Arquivos e verifique a presença do arquivo de texto chamado “mensagem.txt”:

```
┌──(root㉿kali)-[~]
└─# cd /home/aluno/Documentos/Arquivos

┌──(root㉿kali)-[/home/aluno/Documentos/Arquivos]
└─# ls
mensagem.txt
```

3. Em seguida devemos gerar um par de chaves ECC-256, composto por uma chave privada e uma chave pública. No Kali Linux, você pode usar o OpenSSL para isso. Execute o seguinte comando para gerar o par de chaves (verifique os arquivos gerados):

```
┌──(root㉿kali)-[/home/aluno/Documentos/Arquivos]
└─# openssl ecparam -name prime256v1 -genkey -noout -out
chave_privada.pem

┌──(root㉿kali)-[/home/aluno/Documentos/Arquivos]
└─# openssl ec -in chave_privada.pem -pubout -out chave_publica.pem
read EC key
writing EC key

┌──(root㉿kali)-[/home/aluno/Documentos/Arquivos]
└─# ls
chave_privada.pem  chave_publica.pem  mensagem.txt
```

Esses comandos gerarão um arquivo chave_privada.pem contendo a chave privada ECC e um arquivo chave_publica.pem contendo a chave pública ECC.

4. O próximo passo é criptografar o arquivo com a chave pública. Infelizmente o ECC não cifra arquivos diretamente, ele é usado para criar um segredo compartilhado entre duas partes, em vez de criptografar o arquivo diretamente. Veja o procedimento que usaremos para termos a proteção do ECC em cima do AES, que será usado para cifrar o arquivo complementarmente.

```
┌──(root㉿kali)-[/home/aluno/Documentos/Arquivos]
└─# openssl rand -out chave_sessao.bin 32

┌──(root㉿kali)-[/home/aluno/Documentos/Arquivos]
└─# openssl pkeyutl -derive -inkey chave_privada.pem -peerkey
chave_publica.pem -out chave_compartilhada.bin

┌──(root㉿kali)-[/home/aluno/Documentos/Arquivos]
└─# ls
chave_compartilhada.bin  chave_privada.pem  chave_publica.pem
chave_sessao.bin  mensagem.txt

┌──(root㉿kali)-[/home/aluno/Documentos/Arquivos]
└─# openssl enc -aes-256-cbc -salt -in mensagem.txt -out
arquivo_criptografado -pass file:chave_sessao.bin
*** WARNING : deprecated key derivation used.
Using -iter or -pbkdf2 would be better.

┌──(root㉿kali)-[/home/aluno/Documentos/Arquivos]
└─# ls
arquivo_criptografado  chave_compartilhada.bin  chave_privada.pem
chave_publica.pem  chave_sessao.bin  mensagem.txt
```

- O primeiro comando “openssl rand ...” gera uma chave aleatória de 32 bytes (256 bits) e a salva em “chave_sessao.bin”. Essa chave será usada como a chave de sessão para criptografar o arquivo.
- O segundo comando “openssl pkeyutl -derive ...” usa a chave privada e a chave pública para derivar uma chave compartilhada e a salva em “chave_compartilhada.bin”.
- No terceiro comando podemos ver os arquivos criados.
- O quarto comando “openssl enc ...” criptografa o arquivo original usando a chave de sessão compartilhada gerada anteriormente. O arquivo criptografado é salvo em “arquivo_criptografado”. O parâmetro -aes-256-cbc especifica o algoritmo de criptografia AES com uma chave de 256 bits, e -pass file:chave_sessao.bin indica que a chave de criptografia será lida do arquivo “chave_sessao.bin”.

5. Agora você terá o arquivo original criptografado usando a criptografia simétrica AES com uma chave de sessão derivada usando ECC. Lembre-se de que a proteção da chave privada é fundamental para garantir a segurança do sistema. Certifique-se de armazenar a chave privada em um local seguro e de restringir o acesso a ela apenas para usuários autorizados.
6. Finalmente, vamos recuperar a mensagem que está no arquivo "arquivo_criptografado" e salvar o conteúdo descriptografado em "mensagem_recuparada.txt" (**NÃO SE ESQUEÇA DE PRINTAR ESTE PASSO!**):

```
┌──(root㉿kali)-[/home/aluno/Documentos/Arquivos]
└─# openssl enc -d -aes-256-cbc -in arquivo_criptografado -out
mensagem_recuperada.txt -pass file:chave_sessao.bin
*** WARNING : deprecated key derivation used.
Using -iter or -pbkdf2 would be better.

┌──(root㉿kali)-[/home/aluno/Documentos/Arquivos]
└─# ls
arquivo_criptografado    chave_privada.pem  chave_sessao.bin
mensagem.txt
chave_compartilhada.bin  chave_publica.pem  mensagem_recuperada.txt
```

O comando openssl enc é usado novamente, mas agora com a opção -d para indicar que queremos descriptografar o arquivo. O arquivo criptografado é especificado com "-in arquivo_criptografado" e o arquivo descriptografado será salvo como "mensagem_recuparada.txt". A chave de sessão é fornecida com o parâmetro "-pass file:chave_sessao.bin" para realizar a descriptografia corretamente.

7. Lembre-se de que a chave de sessão (chave_sessao.bin) deve ser mantida em segredo e compartilhada de forma segura entre as partes que desejam realizar a descriptografia. A chave privada (chave_privada.pem) também deve ser protegida e mantida em sigilo, pois é essencial para a segurança do sistema.
8. Finalmente, vamos verificar se de fato conseguimos recuperar a mensagem original visualizando o arquivo "mesagem_recuperada.txt".

```
┌──(root㉿kali)-[/home/aluno/Documentos/Arquivos]
└─# cat mensagem_recuperada.txt
Hackers do bem!
```

Confira que realmente a mensagem original foi recuperada!

9. Apague todos os arquivos com exceção do arquivo “mensagem.txt” e posteriormente feche o Terminal:

```
┌──(root㉿kali)-[/home/aluno/Documentos/Arquivos]
└─# rm arquivo_criptografado chave_privada.pem chave_sessao.bin
chave_compartilhada.bin chave_publica.pem mensagem_recuperada.txt

┌──(root㉿kali)-[/home/aluno/Documentos/Arquivos]
└─# ls
mensagem.txt
```

Parabéns! Agora você sabe como usar em conjunto o AES-256 e o ECC-256 para cifrar arquivos com muita maior segurança!

Atividade 8.10 – Implementando criptografia assimétrica para assinatura digital com RSA-2048 em um arquivo no Kali Linux

Nesta atividade, vamos usar o Terminal para assinar digitalmente um arquivo com o algoritmo de criptografia assimétrica RSA de 2048 bits e verificar se a assinatura é válida no Kali Linux.

Vamos começar inicializando nosso Kali Linux via RDP ao IP: 192.168.98.40, com usuário “aluno” e senha “rnpesr”.

1. Abra o Terminal e execute o seguinte comando (com senha “rnpesr”) para ser super usuário:

```
┌──(aluno㉿kali)-[~]
└─$ sudo -i
[sudo] senha para aluno:
```

2. Acesse a pasta Arquivos e verifique a presença do arquivo de texto chamado “mensagem.txt”:

```
┌──(root㉿kali)-[~]
└─# cd /home/aluno/Documentos/Arquivos

┌──(root㉿kali)-[/home/aluno/Documentos/Arquivos]
└─# ls
mensagem.txt
```

3. Em seguida vamos gerar um par de chaves RSA, composto por uma chave privada e uma chave pública. No Kali Linux, você pode usar o OpenSSL para isso. Execute o seguinte comando para gerar o par de chaves (verifique os arquivos criados):

```
┌──(root㉿kali)-[/home/aluno/Documentos/Arquivos]
└─# openssl genpkey -algorithm RSA -out chave_privada.pem -pkeyopt
rsa_keygen_bits:2048
..+..+++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++
+*.....+.+...+..+.............+...........+.......++++++++++++++++++++
++++++++++++++++++++++++++++++++++++++++++*......+.......+.....+...
+..................+.+...+.....+.+..+....+......+.........+.........+..
+.........+.+.....+....+..................+...+..+...............+.
+.....+....+......+..+.........+.+..+.+...........+....+..+.......
+........+...+.+...+...........+....+..+......+.......+.....+......+.
+...+...+..+...+....+...+......+...........+...+.+..+...+....+...+..
+......+...............+.......+..+.+..............+...+...........
+...+...+++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++
++
.....+.+.....+....+...+......+..+.......+.....++++++++++++++++++++++++
++++++++++++++++++++++++++++++++++++++*............+.....+....+..
+.........+......+...+......+.+......+..+............+.+............
+...+..+.+...........+...+.......+........+.+..+...+.+...+.....++++++++
++++++++++++++++++++++++++++++++++++++++++++++++++++*...
+............+......+..+++++++++++++++++++++++++++++++++++++++++++++++
++++++++++++++++
```

```
┌──(root㉿kali)-[/home/aluno/Documentos/Arquivos]
└─# openssl rsa -pubout -in chave_privada.pem -out chave_publica.pem
writing RSA key

┌──(root㉿kali)-[/home/aluno/Documentos/Arquivos]
└─# ls
chave_privada.pem  chave_publica.pem  mensagem.txt
```

Esses comandos gerarão um arquivo “chave_privada.pem” contendo a chave privada RSA de 2048 bits e um arquivo “chave_publica.pem” contendo a chave pública RSA correspondente.

4. Agora vamos assinar o arquivo com a chave privada. Para criar a assinatura digital, você usará a chave privada RSA. Execute o seguinte comando para assinar o arquivo (veja que arquivos foram criados):

```
┌──(root㉿kali)-[/home/aluno/Documentos/Arquivos]
└─# openssl dgst -sha256 -sign chave_privada.pem -out assinatura.bin
mensagem.txt

┌──(root㉿kali)-[/home/aluno/Documentos/Arquivos]
└─# ls
assinatura.bin  chave_privada.pem  chave_publica.pem  mensagem.txt
```

O comando mostrado acima conta com os seguintes parâmetros:

- **-sha256**: Especifica o algoritmo de hash a ser usado, nesse caso, SHA-256. O SHA-256 é um algoritmo de hash criptográfico que gera um resumo (hash) de 256 bits do arquivo ou dados fornecidos.
- **-sign chave_privada.pem**: Indica o arquivo contendo a chave privada RSA a ser usada para assinar o arquivo. A chave privada é necessária para realizar a operação de assinatura digital.
- **-out assinatura.bin**: Especifica o nome do arquivo de saída que conterá a assinatura digital gerada. O arquivo será salvo como “assinatura.bin”.
- **mensagem.txt**: Representa o arquivo original que será assinado.

5. Com esses parâmetros, o comando openssl dgst realizará a operação de assinatura digital do arquivo fornecido usando o algoritmo de hash SHA-256 e a chave privada RSA especificada. A assinatura digital será gerada e salva no arquivo de saída especificado. Após a execução desse comando, a assinatura digital do arquivo será gerada e salva no arquivo “assinatura.bin”.
6. Em seguida verifique se a assinatura criada e representada no arquivo “assinatura.bin” é válida ao corresponder ao arquivo “mensagem.txt”:

```
┌──(root㉿kali)-[/home/aluno/Documentos/Arquivos]
└─# openssl dgst -sha256 -verify chave_publica.pem -signature
assinatura.bin mensagem.txt
Verified OK
```

Os parâmetros utilizados no comando openssl dgst para verificar a assinatura digital são os seguintes:

- **-sha256**: Especifica o algoritmo de hash a ser usado, nesse caso, SHA-256. O SHA-256 é um algoritmo de hash criptográfico que gera um resumo (hash) de 256 bits do arquivo ou dados fornecidos.
- **-verify chave_publica.pem**: Indica o arquivo contendo a chave pública RSA a ser usada para verificar a assinatura. A chave pública é necessária para realizar a operação de verificação da assinatura digital.
- **-signature assinatura.bin**: Especifica o nome do arquivo que contém a assinatura digital a ser verificada. O arquivo deve ser um arquivo binário que contém a assinatura gerada anteriormente.
- **mensagem.txt**: Representa o arquivo original que foi assinado digitalmente.

Conforme mostrado no comando anterior, a assinatura digital é válida!

7. Para consolidar nossa aprendizagem, vamos explorar o que acontece se tentarmos validar um arquivo diferente. Crie outro arquivo de texto:

```
┌──(root㉿kali)-[/home/aluno/Documentos/Arquivos]
└─# nano mensagem_errada.txt
```

8. Insira o seguinte texto:

```
Hackers do mal!
```

Em seguida, aperte “Ctrl + x”, “s” para salvar e “Enter” para confirmar. Neste momento, acabamos de criar o arquivo “mensagem.txt”.

9. Finalmente, vamos tentar validar o arquivo “mensagem_errada.txt” com a assinatura digital (usando a chave pública do criador) criada anteriormente (**NÃO SE ESQUEÇA DE PRINTAR ESTE PASSO!**):

```
┌──(root㉿kali)-[/home/aluno/Documentos/Arquivos]
└─# openssl dgst -sha256 -verify chave_publica.pem -signature
assinatura.bin mensagem_errada.txt
Verification failure
4047DDE3B77F0000:error:02000068:rsa routines:ossl_rsa_verify:bad
signature:../crypto/rsa/rsa_sign.c:430:
4047DDE3B77F0000:error:1C880004:Provider routines:rsa_verify:RSA
lib:../providers/implementations/signature/rsa_sig.c:774
```

Note que o arquivo “mensagem_errada.txt” apresentou erro na validação. Isso significa que a assinatura digital presente no arquivo “assinatura.bin”, não é correspondente a esse arquivo txt.

10. Apague todos os arquivos da pasta “Arquivos” e posteriormente feche o Terminal:

```
┌──(root㉿kali)-[/home/aluno/Documentos/Arquivos]
└─# rm -r *
zsh: sure you want to delete all 5 files in /home/aluno/Documentos/
Arquivos [yn]? y

┌──(root㉿kali)-[/home/aluno/Documentos/Arquivos]
└─# ls
```

Parabéns! Agora você sabe como implementar assinaturas digitais com o algoritmo RSA-2048!